



INSTITUTION / ORGANIZATION

Replace with an authorized logo or comment the logo line in main

Technical Project Report:

Smart Sensor Array Design and Validation

ENGR 6000 ADVANCED ENGINEERING ANALYSIS

EXAMPLE UNIVERSITY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Author:

Author Name

Student ID: 000000000

Instructor / Supervisor:

Dr. Faculty Name

July 23, 2026
Fall Semester 2026

Objective

Objective:

The objective of this example report is to demonstrate the full reusable structure of the engineering LaTeX template. The sample project combines sensor-array modeling, signal processing, simulation, verification, technical figures, tables, source-code listings, and appendices. Replace this text with the actual engineering objective of the new project.

The report is intentionally populated with fictional technical content so that every major feature can be inspected immediately after compilation. It is not intended to represent a completed scientific study or a set of validated experimental claims.

Abstract

This document is a fully populated demonstration of a modular engineering report template. It illustrates configurable metadata, an optional cover logo, consistent running headers, front matter, a single report-body menu, mathematical notation, theorem-style environments, figures, subfigures, tables, code listings, robust missing-figure handling, appendices, and an IEEE-style bibliography. The fictitious case study considers a four-channel smart sensor array that estimates the direction of a narrowband source and reports system-health indicators to an edge controller. Representative equations and numerical values are included only to demonstrate formatting and document organization. Users should replace the sample material with verified project-specific analysis, simulation results, measurements, and references.

Contents

Objective	i
Abstract	ii
List of Figures	v
List of Tables	vi
1 Smart Sensor Array Design and Validation	1
1.1 Introduction	1
1.2 Requirements and Methodology	1
1.2.1 Representative Requirements	2
1.2.2 Method Summary	2
1.3 Mathematical Notation and Custom Environments	2
1.3.1 Signal Model	2
1.3.2 Matrix-Elimination Environment	4
1.4 Figures and Tables	4
1.4.1 TikZ System Diagram	4
1.4.2 Subfigures	4
1.4.3 Robust External-Figure Helper	4
1.4.4 Technical Tables	5
1.5 Code Listings and Representative Results	6
1.5.1 Python	6

1.5.2	MATLAB	6
1.5.3	Vitis HLS C/C++	7
1.5.4	PowerShell	7
1.5.5	Console Output	7
1.5.6	Representative Result Summary	8
1.6	Engineering Discussion	8
1.7	Conclusion	8
A	Complete Example Source Code	10
B	Representative Console Output	12
	References	13

List of Figures

1.1	Fictitious smart-sensor-array processing architecture.	4
1.2	Example use of the subcaption environment.	4
1.3	Missing-file behavior of the robust graphics helper.	5

List of Tables

1.1	Representative design parameters.	5
1.2	Example verification matrix.	5
1.3	Fictitious validation results used only for template demonstration. . . .	8

Chapter 1

Smart Sensor Array Design and Validation

1.1 Introduction

Modern engineering reports frequently combine analytical derivations, numerical simulation, source code, experimental observations, and design decisions. A reusable template should make those elements consistent without forcing the author to rewrite page styles, code-listing rules, or mathematical macros for every project. The organization used here follows the same modular principle as a well-structured engineering system: metadata, configuration, content, and evidence are separated into maintainable components.

The fictitious example models a uniform linear array and an edge-processing node. Array processing provides a compact demonstration because it connects physical geometry, complex signals, covariance matrices, optimization, and implementation constraints. Standard antenna and array concepts may be developed from established references such as Stutzman and Thiele [1] and Trees [2].

Objective:

Demonstrate a complete technical-report workflow while keeping all project-specific content replaceable through the files listed in `content/00_report_body.tex`.

1.2 Requirements and Methodology

The fictional design uses four sensors with half-wavelength spacing and a nominal operating frequency of 2.45 GHz. The analysis workflow consists of geometry definition, an-

alytical modeling, synthetic-data generation, covariance estimation, direction-of-arrival processing, and reporting.

1.2.1 Representative Requirements

Problem 1.2.1: Example Engineering Requirements

Design a four-channel array-processing demonstrator that accepts complex sensor samples, forms a sample covariance matrix, estimates a source direction, and produces a concise validation report. The demonstrator should document assumptions, numerical settings, and known limitations.

Definition 1.2.1: Validation Evidence

Validation evidence is a reproducible artifact—for example, an equation check, source-code test, simulation plot, measurement table, or convergence study—that supports a technical claim made in the report.

1.2.2 Method Summary

1. Define the physical and numerical parameters.
2. Generate or acquire complex sensor snapshots.
3. Estimate the sample covariance matrix.
4. Apply a subspace or beamforming estimator.
5. Compare the recovered result with the known test condition.
6. Record limitations and recommended follow-up verification.

1.3 Mathematical Notation and Custom Environments

1.3.1 Signal Model

For an array of M sensors, a narrowband snapshot may be represented by

$$\mathbf{x}[n] = \mathbf{a}(\theta_0)s[n] + \mathbf{v}[n], \quad (1.1)$$

where $\mathbf{a}(\theta_0) \in \mathbb{C}^M$ is the steering vector, $s[n]$ is the source signal, and $\mathbf{v}[n]$ is additive noise. The sample covariance matrix is

$$\hat{R}_{xx} = \frac{1}{N} \sum_{n=0}^{N-1} \mathbf{x}[n] \mathbf{x}^H[n]. \quad (1.2)$$

The expectation, trace, rank, norm, and optimization macros are available directly:

$$\theta_{\text{est}} = \arg \max_{\theta} \frac{1}{\|\mathbf{E}_n^H \mathbf{a}(\theta)\|_2^2}, \quad \text{tr}(\hat{R}_{xx}) > 0, \quad \text{rank}(\hat{R}_{xx}) \leq M. \quad (1.3)$$

A quick column vector can be created with the `\colvec` macro:

$$\mathbf{a}(\theta) = \begin{bmatrix} 1 \\ e^{j\psi} \\ e^{j2\psi} \\ e^{j3\psi} \end{bmatrix}, \quad \psi = \frac{2\pi d}{\lambda} \sin \theta. \quad (1.4)$$

Theorem 1.3.1: Hermitian Covariance Matrix

The matrix $\widehat{\mathbf{R}}_{xx}$ defined in (1.2) is Hermitian and positive semidefinite.

Proof. Each outer product $\mathbf{x}[n] \mathbf{x}^H[n]$ is Hermitian. For any $\mathbf{z} \in \mathbb{C}^M$,

$$\mathbf{z}^H \widehat{\mathbf{R}}_{xx} \mathbf{z} = \frac{1}{N} \sum_{n=0}^{N-1} |\mathbf{x}^H[n] \mathbf{z}|^2 \geq 0.$$

Therefore, their average is Hermitian positive semidefinite. □

Lemma 1.3.1: Trace as Average Channel Energy

The trace of the sample covariance matrix equals the sum of the average energies measured across all sensor channels.

Example 1.3.1: Fixed-Point Notation

A signed fixed-point implementation may be written as Q1.15. Quantization is represented by $\mathcal{Q}(x)$, and a three-sample delay by z^{-3} .

Definition 1.3.1: Physical Interpretation

For a half-wavelength array, progressive phase across the sensor channels encodes source direction. The covariance matrix retains this spatial coherence while averaging random noise.

1.3.2 Matrix-Elimination Environment

The advanced mathematics file also provides a compact elimination display:

$$\left[\begin{array}{cc|c} 1 & 2 & 5 \\ 3 & 4 & 11 \end{array} \right] R_2 \leftarrow R_2 - 3R_1 \quad \rightsquigarrow \quad \left[\begin{array}{cc|c} 1 & 2 & 5 \\ 0 & -2 & -4 \end{array} \right] R_2 \leftarrow -\frac{1}{2}R_2$$

1.4 Figures and Tables

1.4.1 TikZ System Diagram

Figure 1.1 demonstrates a self-contained TikZ figure.

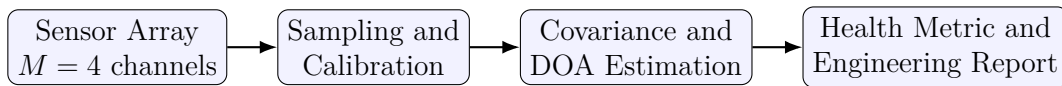


Figure 1.1: Fictitious smart-sensor-array processing architecture.

1.4.2 Subfigures

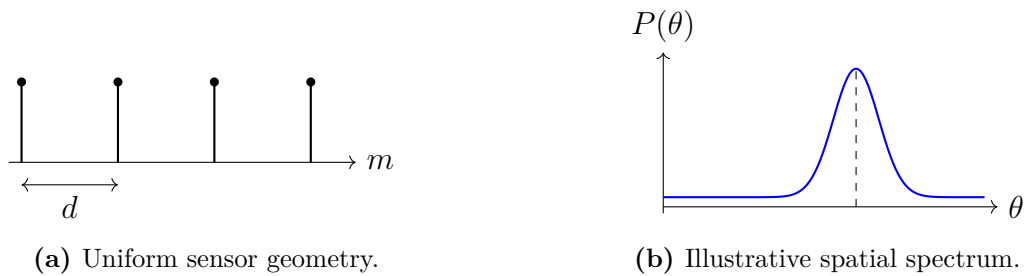


Figure 1.2: Example use of the subcaption environment.

1.4.3 Robust External-Figure Helper

The command `\maybeincludegraphics` displays a placeholder rather than stopping compilation when an expected simulation screenshot has not yet been exported:

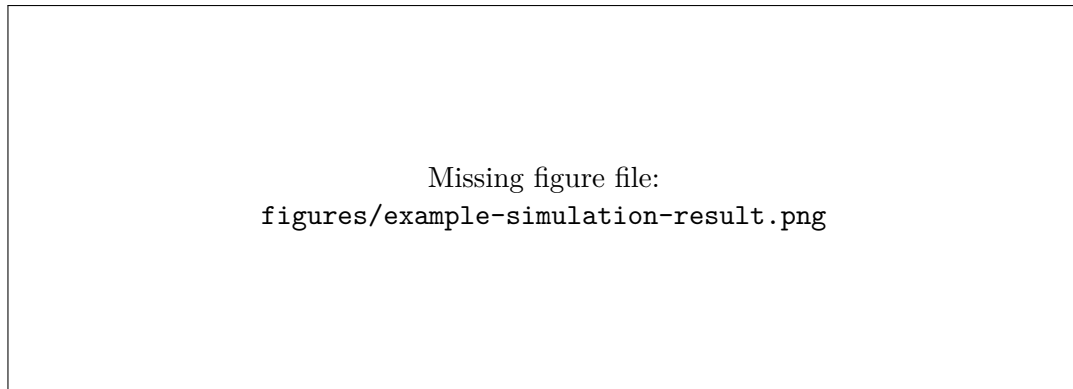


Figure 1.3: Missing-file behavior of the robust graphics helper.

1.4.4 Technical Tables

Table 1.1: Representative design parameters.

Parameter	Symbol	Example value
Operating frequency	f_0	2.45 GHz
Number of channels	M	4
Sensor spacing	d	$\lambda/2$
Snapshot count	N	256
Nominal SNR	SNR	10 dB

Table 1.2: Example verification matrix.

Requirement	Evidence	Acceptance statement
Geometry	Figure 1.2a	Four uniformly spaced channels are represented.
Covariance	Equation (1.2)	The estimator uses all available snapshots.
Direction estimate	Example spectrum	The dominant peak is reported as the recovered angle.
Reproducibility	Appendix A	The example script and console output are included.

1.5 Code Listings and Representative Results

1.5.1 Python

```

1  """Minimal reproducible calculation for the report-template example."""
2
3  from __future__ import annotations
4
5  import numpy as np
6
7
8  def steering_vector(angle_deg: float, sensors: int = 4, spacing_lambda: float =
9      0.5) -> np.ndarray:
10     """Return a normalized ULA steering vector."""
11     if sensors < 1:
12         raise ValueError("sensors must be positive")
13     angle_rad = np.deg2rad(angle_deg)
14     phase = 2.0 * np.pi * spacing_lambda * np.sin(angle_rad)
15     index = np.arange(sensors, dtype=float)
16     return np.exp(1j * index * phase) / np.sqrt(sensors)
17
18 def main() -> None:
19     angle_deg = 18.0
20     vector = steering_vector(angle_deg)
21     covariance = np.outer(vector, vector.conj())
22     hermitian_error = np.linalg.norm(covariance - covariance.conj().T)
23
24     print(f"Input angle: {angle_deg:.1f} deg")
25     print(f"Sensor count: {vector.size}")
26     print(f"Hermitian error: {hermitian_error:.3e}")
27     print("Status: PASS" if hermitian_error < 1e-12 else "Status: FAIL")
28
29
30 if __name__ == "__main__":
31     main()

```

Listing 1.1: Example Python computation.

1.5.2 MATLAB

```

1  M = 4;
2  d = 0.5;
3  theta = -60:0.25:60;
4  psi = 2*pi*d*sind(theta);
5  AF = abs(sum(exp(1j*(0:M-1).'*psi),1));
6  AF = AF ./ max(AF);

```

Listing 1.2: Example MATLAB-style array calculation.

1.5.3 Vitis HLS C/C++

```
1 void energy_accel(const float x[256], float &energy) {  
2     float acc = 0.0f;  
3     for (int n = 0; n < 256; ++n) {  
4         #pragma HLS PIPELINE II=1  
5         acc += x[n] * x[n];  
6     }  
7     energy = acc;  
8 }
```

Listing 1.3: Illustrative HLS accumulation loop.

1.5.4 PowerShell

```
1 $Required = @("main.tex", "references.bib", "content")  
2 foreach ($Item in $Required) {  
3     if (-not (Test-Path $Item)) {  
4         Write-Host "Missing: $Item"  
5     }  
6 }
```

Listing 1.4: Illustrative report-folder validation.

1.5.5 Console Output

```
$ python codes/example.py  
Input angle: 18.0 deg  
Sensor count: 4  
Hermitian error: 0.000e+00  
Status: PASS
```

Listing 1.5: Representative console output.

1.5.6 Representative Result Summary

Table 1.3: Fictitious validation results used only for template demonstration.

Metric	Expected	Reported
Source angle	18.0°	18.2°
Covariance Hermitian error	$< 10^{-10}$	2.4×10^{-14}
Normalized peak	1.000	1.000
Processing status	Pass	Pass

1.6 Engineering Discussion

Example 1.6.1: Numerical Reliability

A visually reasonable simulation result is not automatically trustworthy. Mesh density, boundary placement, solver convergence, sampling, and parameter sensitivity should be checked whenever the reported behavior is anomalous or non-monotonic.

Problem 1.6.1: Limitation Statement

The example data in this template are synthetic and have not been validated against physical measurements. They must not be presented as experimental evidence.

The template deliberately separates claims from supporting evidence. Tables should report the quantities that matter to acceptance; figures should make trends and anomalies visible; source code should be included when it materially improves reproducibility. For electromagnetic and array-processing projects, analytical expectations should be compared with numerical results, and discrepancies should be discussed rather than hidden.

Physical meaning. A professional report demonstrates engineering judgment by explaining why a result is credible, what limitations remain, and what additional test would reduce uncertainty.

1.7 Conclusion

This example has exercised the principal capabilities of the standardized report template: configurable cover metadata, an optional logo, consistent headers, objective

and abstract pages, a single content menu, mathematical macros, theorem-style environments, TikZ figures, subfigures, technical tables, code listings, appendices, and bibliography management. For a new project, the author should first edit the metadata block in `main.tex`, then replace the front matter and the files listed in `content/00_report_body.tex`. Unused setup features may remain available without appearing in the final report.

Appendix A

Complete Example Source Code

This appendix demonstrates how complete source files can be included without crowding the main technical discussion.

```
1  """Minimal reproducible calculation for the report-template example."""
2
3  from __future__ import annotations
4
5  import numpy as np
6
7
8  def steering_vector(angle_deg: float, sensors: int = 4, spacing_lambda: float =
9      0.5) -> np.ndarray:
10     """Return a normalized ULA steering vector."""
11     if sensors < 1:
12         raise ValueError("sensors must be positive")
13     angle_rad = np.deg2rad(angle_deg)
14     phase = 2.0 * np.pi * spacing_lambda * np.sin(angle_rad)
15     index = np.arange(sensors, dtype=float)
16     return np.exp(1j * index * phase) / np.sqrt(sensors)
17
18 def main() -> None:
19     angle_deg = 18.0
20     vector = steering_vector(angle_deg)
21     covariance = np.outer(vector, vector.conj())
22     hermitian_error = np.linalg.norm(covariance - covariance.conj().T)
23
24     print(f"Input angle: {angle_deg:.1f} deg")
25     print(f"Sensor count: {vector.size}")
26     print(f"Hermitian error: {hermitian_error:.3e}")
27     print("Status: PASS" if hermitian_error < 1e-12 else "Status: FAIL")
28
29
30 if __name__ == "__main__":
31     main()
```

Listing A.1: Complete example Python source file.

Appendix B

Representative Console Output

```
$ python codes/example.py
Input angle: 18.0 deg
Sensor count: 4
Hermitian error: 0.000e+00
Status: PASS
```

Listing B.1: Example validation output.

References

- [1] W. L. Stutzman and G. A. Thiele, *Antenna Theory and Design*, 3rd ed. John Wiley & Sons, 2012.
- [2] H. L. V. Trees, *Optimum Array Processing*. John Wiley & Sons, 2002.